Aide — entraînement du U-Net 3D

GUI_train.py, train.py (étage 1, positions), train_stage2.py (étage 2, instances) — état du 2026-10-22.

Déroulement

- **1. Jeu de données** : générer un jeu avec GUI_generate_dataset.py (HELP_GUI_GENERATE_DATASET.pdf), ou reprendre un dossier de datasets/.
- **2. Étage 1** (positions) : environ 30 à 45 min pour 6000 itérations ; donne le modèle de détection.
- **3. Étage 2** (instances) : environ 55 min ; même décimation que l'étage 1 avec lequel il sera utilisé.
- **4. Utiliser** : choisir les dossiers des deux runs dans GUI_inference.py (onglet Modèles), ou les copier à la place de models/stage1 et models/stage2 pour en faire les modèles par défaut.

Lancer

`python deep_learning_segmentation/GUI_train.py` ; en ligne de commande : `python deep_learning_segmentation/train.py --name s1 --train_dir <jeu>/train --val_dir <jeu>/val` et `python deep_learning_segmentation/train_stage2.py --name s2 --data <jeu>`.

Boutons et affichage

- **Générer un jeu…** : ouvre GUI_generate_dataset.py dans une fenêtre indépendante.
- **Entraîner l'étage 1 / l'étage 2** : lance le script dans un processus séparé (priorité basse, l'interface reste réactive), avec les réglages des onglets ; demande avant d'écraser un run existant.
- **Arrêter** : arrêt propre (fichier STOP dans le dossier du run) : fin de l'itération, dernière validation, bilan enregistré.
- **Tuer** : arrêt immédiat (le dernier model_best.pt enregistré reste valable).
- **Suivre un run…** : reprend l'affichage d'un run en cours ou fini (son train.log).
- **Courbes** : loss d'entraînement et validation (étage 1 : F1, rappel, rappel des couchées ; étage 2 : IoU, part d'IoU > 0,5, rappel et précision des centres) ; journal et progression en bas.
- **Fermer** pendant un entraînement : le laisser tourner (suivi possible ensuite) ou l'arrêter proprement. Réglages mémorisés dans GUI_train_params.json.

Contenu d'un run (runs/<nom>/)

Fichier	Contenu
`model_best.pt`	poids du meilleur modèle en validation + toute la configuration (réglages, grille, optique, filtre, statistiques du film…) + seuil retenu (étage 1)
`config.json`	la même configuration, lisible
`history.json`	une ligne par validation (loss, métriques, seuil, temps)
`train_summary.json`	bilan : itérations, temps, mémoire GPU, meilleure validation
`train.log`	journal complet du script

Onglet Données

Champ	Défaut	Option du script	Explication
Jeu de données	deep_learning_segmentation/datasets/dataset_sim_200um_N20-400	`--train_dir / --val_dir (étage 1), --data (étage 2)`	Dossier produit par GUI_generate_dataset.py (parameters.yaml, train/, val/, test/), utilisé par les deux étages ; un dossier de .npz de generate_dataset.py (train/, val/) est aussi accepté — dossier
Profondeur min reconstruite (µm)	—	`--z_min_um`	Début du volume reconstruit (distance au capteur), deux étages ; vide : distance volume-caméra du jeu (0 pour un jeu .npz) — texte
Profondeur max reconstruite (µm)	—	`--z_max_um`	Fin du volume reconstruit ; vide : distance volume-caméra + nombre de plans × pas du jeu (207 pour un jeu .npz) — texte

Sous les champs : résumé du jeu (parameters.yaml) et volume reconstruit retenu (automatique ou saisi). Les défauts des onglets Étage 1 et Étage 2 sont les constantes en tête de train.py et train_stage2.py.

Onglet Étage 1

Champ	Défaut	Option du script	Explication
Nom du run	stage1	`--name`	Sous-dossier créé dans le dossier des runs (model_best.pt, config.json, history.json, train.log) — texte
Dossier des runs	deep_learning_segmentation/runs	`--out`	dossier
Loss	centernet	`--loss`	centernet : carte des centres, positions = maxima (la meilleure au benchmark) ; les autres apprennent un masque grossier de la bactérie — choix : centernet, wmse, bce_dice, focal_tversky
Normalisation	filmmad	`--norm`	Mise à l'échelle du volume reconstruit (Re, Im) avant le réseau : (V − centre) / σ. filmmad : médiane et 1,4826 × MAD du film (médianes sur ses hologrammes), les mêmes pour toutes ses images ; globalmad : celles du volume de chaque hologramme ; filmstd, globalstd : idem avec moyenne et écart-type ; localmad : par maille (taille ci-dessous), interpolées ; fixed : échelle calculée du bruit capteur (modèles du 2026-10-18) — choix : filmmad, globalmad, filmstd, globalstd, localmad, fixed
Hologrammes pour les statistiques du film	32	`--film_holos`	filmmad, filmstd : nombre d'hologrammes, régulièrement espacés, sur lesquels le centre et σ sont mesurés — entier
Maille MAD locale, Z (µm)	13	`--mad_mesh_um (Z)`	localmad seulement ; HOLOTRACKER : 12,5 µm — nombre
Maille MAD locale, XY (µm)	11.2	`--mad_mesh_um (XY)`	localmad seulement ; arrondie à 4 voxels ; HOLOTRACKER : 11,2 µm — nombre
Décimation latérale	2	`--decim`	Décimation latérale du volume grossier : voxel XY = pixel objet × décimation (×2 : 0,35 µm), par recadrage du spectre, sans perte si le passe-bas est ≤ taille / (2 × décimation) (256 px pour 1024 et ×2). ×1 : 4 fois plus de calcul ; ×4 : 4 fois moins, passe-bas ≤ 128. Les deux étages doivent avoir la même — choix : 1, 2, 4
Passe-haut (px de Fourier)	15	`--high_pass`	Filtre passe-haut de la reconstruction : rayon (px de Fourier du capteur) sous lequel le spectre est coupé (fond lentement variable, résidus d'éclairage) ; 0 : aucun — nombre
Passe-bas (px de Fourier)	250	`--low_pass`	Filtre passe-bas : rayon au-delà duquel le spectre est coupé (bruit haute fréquence) ; 0 : aucun — nombre
Pas Z (µm)	1	`--z_step`	Pas des plans du volume grossier ; 1 µm ; 2 µm : réseau 2,3× plus rapide, σz 0,28 au lieu de 0,16 µm — nombre
Patchs (Z Y X, voxels)	48 96 96	`--patch`	Taille des blocs du volume grossier vus à chaque pas d'entraînement (multiples de 2^niveaux) ; 48 96 96 = 48 × 33,6 × 33,6 µm à ×2 ; 64 128 128 déborde des 6 Go — texte
Patchs centrés sur une bactérie (part)	0.7	`--pos_fraction`	Part des patchs tirés autour d'une bactérie ; les autres sont tirés au hasard (le volume est presque vide) — nombre
Largeur des cibles XY (µm)	0.5	`--heat_sigma_xy_um`	Cible : chaque bactérie est une gaussienne de cet écart-type dans la carte des centres ; plus large : apprentissage plus facile, mais pics de deux voisines confondus — nombre
Largeur des cibles Z (µm)	1	`--heat_sigma_z_um`	Idem en Z (la réponse est ≈ 2,5 fois plus longue en Z) — nombre
Fenêtre des maxima (Z Y X, voxels)	3 5 5	`--peak_box`	Une détection = un maximum local de la carte des centres dans cette fenêtre (valeurs impaires, voxels grossiers) ; sert à la validation et devient le défaut de l'inférence — texte

Champ	Défaut	Option du script	Explication
Canaux de base	16	`--base`	Canaux du premier niveau, doublés à chaque niveau ; 16 (1,4 M paramètres) ; 8 : 4× plus petit — entier
Niveaux du U-Net	3	`--depth`	Sous-échantillonnages par 2 ; 3 : champ récepteur de plusieurs dizaines de voxels — entier
Itérations	6000	`--iters`	Pas d'optimisation ; ~0,3 à 0,45 s chacun (Quadro RTX 3000) — entier
Taux d'apprentissage	0.001	`--lr`	Maximum (AdamW) : montée linéaire sur 200 itérations puis cosinus — nombre
Tête d'orientation	non	`--orient_weight 1`	Prédit aussi l'axe de la bactérie (tenseur $a \cdot a^T$), sans coût mesurable — case à cocher

Onglet Étage 2

Champ	Défaut	Option du script	Explication
Nom du run	stage2	`--name`	Sous-dossier créé dans le dossier des runs — texte
Dossier des runs	deep_learning_segmentation/runs	`--out`	dossier
Décimation latérale	2	`--decim`	Décimation latérale du volume grossier : voxel XY = pixel objet × décimation (×2 : 0,35 µm), par recadrage du spectre, sans perte si le passe-bas est ≤ taille / (2 × décimation) (256 px pour 1024 et ×2). ×1 : 4 fois plus de calcul ; ×4 : 4 fois moins, passe-bas ≤ 128. Les deux étages doivent avoir la même — choix : 1, 2, 4
Passe-haut (px de Fourier)	15	`--high_pass`	Filtre passe-haut de la reconstruction : rayon (px de Fourier du capteur) sous lequel le spectre est coupé (fond lentement variable, résidus d'éclairage) ; 0 : aucun — nombre
Passe-bas (px de Fourier)	250	`--low_pass`	Filtre passe-bas : rayon au-delà duquel le spectre est coupé (bruit haute fréquence) ; 0 : aucun — nombre
Pas Z du volume grossier (µm)	1	`--z_step`	Plans du volume grossier d'où partent les fenêtres (le plan le plus proche est refocalisé) — nombre
Fenêtre fine : plans	32	`--fine_nz`	Hauteur de la fenêtre autour de chaque détection, en plans fins (pas des masques du jeu, 0,25 µm : 32 = 8 µm) ; multiple de 2^niveaux — entier
Fenêtre fine : côté (voxels)	40	`--fine_nxy`	Côté de la fenêtre en pixels objet (0,175 µm : 40 = 7 µm) ; multiple de 2^niveaux — entier
Carré refocalisé (px grossiers)	64	`--refocus_crop`	Région du plan grossier propagée vers la fenêtre fine (64 px à ×2 = 22,4 µm) : assez grande pour contenir la lumière qui s'étale sur la hauteur de la fenêtre — entier
Largeur des cibles des centres XY (µm)	0.25	`--heat_sigma_xy_um`	Cible : σ des gaussiennes de la carte fine des centres (second tour) — nombre
Largeur des cibles des centres Z (µm)	0.5	`--heat_sigma_z_um`	nombre
Attribution d'une fenêtre, XY (µm)	1	`--tol_xy_um`	Une fenêtre appartient à la bactérie la plus proche de son centre si elle est à moins de cette distance (XY et Z normalisés) ; sinon c'est une fausse détection (masque vide) — nombre
Attribution d'une fenêtre, Z (µm)	2	`--tol_z_um`	nombre
Itérations	6000	`--iters`	~0,55 s par itération — entier
Taux d'apprentissage	0.001	`--lr`	Maximum (AdamW) — nombre
Canaux de base	16	`--base`	entier
Niveaux du U-Net	3	`--depth`	entier
Fenêtres par hologramme	16	`--crops_per_holo`	Fenêtres tirées dans chaque hologramme reconstruit — entier
Hologrammes par pas	2	`--holos_per_step`	Hologrammes reconstruits par pas (fenêtres par pas = produit des deux) — entier
Décalage des fenêtres XY (µm)	0.25	`--jitter_xy`	σ du décalage aléatoire : erreur de position simulée de l'étage 1 — nombre
Décalage des fenêtres Z (µm)	0.5	`--jitter_z`	nombre
Fenêtres négatives (part)	0.1	`--neg_fraction`	Au hasard dans le volume : apprend à rejeter les fausses détections — nombre
Fenêtres entre deux voisines (part)	0.15	`--mid_fraction`	Au milieu de deux bactéries proches : ce que voit l'étage 2 quand l'étage 1 les a fusionnées — nombre
Voisines « proches » à moins de (µm)	3	`--mid_pair_um`	Distance entre centres définissant ces paires — nombre

Champ	Défaut	Option du script	Explication
Voxels minimum d'une instance	20	`--vmin`	En dessous : fausse détection (≈ 60 voxels pour une bactérie de 1,8 × 0,6 µm) ; repris à l'inférence — entier

train.py — étage 1 (positions)

Constantes en tête du script (valeurs par défaut des options) : DECIM, HIGH_PASS, LOW_PASS, Z_STEP_UM, PATCH, POS_FRACTION, HEAT_SIGMA_XY_UM, HEAT_SIGMA_Z_UM, PEAK_BOX, FILM_STATS_HOLOS, MAD_MESH_UM.

Option	Défaut	Rôle
`--name`	obligatoire	nom du run (sous-dossier de --out)
`--loss {bce_dice, focal_tversky, centernet, wmse}`	centernet	centernet : carte des centres (retenue) ; wmse, bce_dice, focal_tversky : masque grossier
`--input {reim, abs}`	reim	reim : Re et Im du champ ; abs : module
`--norm {globalmad, filmmad, globalstd, filmstd, localmad, fixed}`	fixed	globalmad : MAD globale par hologramme : (V − médiane) / (1,4826 × MAD) du volume de l'hologramme; filmmad : MAD globale du film : médiane et σ du film (médianes sur ses hologrammes), les mêmes pour toutes ses images; globalstd : moyenne et écart-type du volume de chaque hologramme : (V − moyenne) / écart-type; filmstd : moyenne et écart-type du film (médianes sur ses hologrammes), les mêmes pour toutes ses images; localmad : MAD locale : médiane et σ par maille, interpolées (seuil local MAD d'HOLOTRACKER); fixed : échelle fixe (modèles du 2026-10-18) : σ calculé du bruit capteur 0,015 et de la bande du filtre
`--mad_mesh_um MAD_MESH_UM MAD_MESH_UM`	13 11.2	maille de la MAD locale (µm) : Z puis XY (HOLOTRACKER : 12,5 × 11,2)
`--film_holos`	32	hologrammes utilisés pour les statistiques d'une normalisation « film »
`--decim {1, 2, 4}`	2	décimation latérale du volume reconstruit (voxel XY = pixel objet × décimation) ; l'étage 2 doit être entraîné avec la même
`--high_pass`	15	filtre passe-haut de la reconstruction (rayon en px de Fourier du capteur, 0 : aucun)
`--low_pass`	250	filtre passe-bas (0 : aucun) ; sans perte à la décimation si ≤ taille / (2 × décimation)
`--heat_sigma_xy_um`	0.5	cible : largeur σ des gaussiennes de la carte des centres en XY (µm)
`--heat_sigma_z_um`	1	idem en Z (µm)
`--peak_box PEAK_BOX PEAK_BOX PEAK_BOX`	3 5 5	fenêtre des maxima locaux de la carte des centres (Z, Y, X), voxels grossiers impairs
`--z_step`	1	pas Z de la grille (µm)
`--z_min_um`	—	début du volume reconstruit (µm) ; défaut : distance volume-caméra du jeu (0 pour un jeu .npz)
`--z_max_um`	—	fin du volume reconstruit (µm) ; défaut : distance + plans × pas du jeu (207 pour un jeu .npz)
`--depth_um`	—	ancien réglage : volume de 0 à depth_um
`--orient_weight`	0	> 0 : tête d'orientation (cible chaleur)
`--train_dir`	deep_learning_segmentation/datasets/dataset_sim_200um_N20-400/train	dossier train/ d'un jeu (GUI_generate_dataset.py ou .npz de generate_dataset.py)
`--val_dir`	deep_learning_segmentation/datasets/dataset_sim_200um_N20-400/val	dossier val/ du même jeu
`--iters`	4000	itérations (pas d'optimisation)
`--lr`	0.001	taux d'apprentissage maximal (AdamW)
`--warmup`	200	itérations de montée linéaire du taux, puis cosinus
`--base`	16	canaux du premier niveau (doublés à chaque niveau)
`--depth`	3	niveaux du U-Net (sous-échantillonnages par 2)
`--patch PATCH PATCH PATCH`	48 96 96	patchs (Z, Y, X) en voxels grossiers ; 4 patchs de 64 × 128 × 128 débordent des 6 Go
`--norm_layer {batch, group}`	batch	couches de normalisation du réseau

Option	Défaut	Rôle
`--holos_per_step`	2	hologrammes reconstruits par pas
`--patches_per_holo`	2	patchs tirés de chaque hologramme
`--pos_fraction`	0.7	part des patchs centrés près d'une bactérie
`--eval_every`	500	validation toutes les N itérations
`--val_limit`	24	hologrammes de validation utilisés
`--seed`	0	graine (tirages des patchs, initialisation)
`--out`	deep_learning_segmentation/runs	dossier des runs

train_stage2.py — étage 2 (instances)

Constantes en tête du script : DECIM, HIGH_PASS, LOW_PASS, Z_STEP_UM, FINE_NZ, FINE_NXY, REFOCUS_CROP, HEAT_SIGMA_XY_UM, HEAT_SIGMA_Z_UM, TOL_XY_UM, TOL_Z_UM, CROPS_PER_HOLO, NEG_FRACTION, MID_FRACTION, MID_PAIR_UM, JITTER_XY_UM, JITTER_Z_UM, VMIN.

Option	Défaut	Rôle
`--name`	obligatoire	nom du run (sous-dossier de --out)
`--data`	deep_learning_segmentation/datasets/dataset_sim_200um_N20-400	dossier d'un jeu avec masques d'instance (train/, val/)
`--z_min_um`	—	début du volume grossier (µm) ; défaut : distance volume-caméra du jeu (0 pour un jeu .npz)
`--z_max_um`	—	fin du volume grossier (µm) ; défaut : distance + plans × pas du jeu (207 pour un jeu .npz)
`--out`	deep_learning_segmentation/runs	dossier des runs
`--decim {1, 2, 4}`	2	décimation latérale du volume grossier ; doit être celle de l'étage 1 utilisé avec ce modèle
`--high_pass`	15	filtre passe-haut (px de Fourier du capteur)
`--low_pass`	250	filtre passe-bas (px de Fourier du capteur)
`--z_step`	1	pas Z du volume grossier (µm)
`--fine_nz`	32	fenêtre fine : nombre de plans (pas = pas des masques du jeu) ; multiple de 2^niveaux
`--fine_nxy`	40	fenêtre fine : côté en voxels fins (pixel objet) ; multiple de 2^niveaux
`--refocus_crop`	64	carré du volume grossier refocalisé autour de chaque fenêtre (px grossiers, pair)
`--heat_sigma_xy_um`	0.25	cible : σ des gaussiennes de la carte fine des centres en XY (µm)
`--heat_sigma_z_um`	0.5	idem en Z (µm)
`--tol_xy_um`	1	attribution d'une fenêtre à la bactérie la plus proche : tolérance XY (µm)
`--tol_z_um`	2	idem en Z (µm)
`--iters`	6000	itérations (pas d'optimisation)
`--lr`	0.001	taux d'apprentissage maximal (AdamW)
`--warmup`	200	itérations de montée linéaire du taux, puis cosinus
`--holos_per_step`	2	hologrammes reconstruits par pas
`--crops_per_holo`	16	fenêtres tirées de chaque hologramme
`--neg_fraction`	0.1	part des fenêtres au hasard (fausses détections)
`--mid_fraction`	0.15	part des fenêtres au milieu de deux voisines proches (détection fusionnée)
`--mid_pair_um`	3	distance maximale de deux « voisines proches » (µm)
`--jitter_xy`	0.25	décalage aléatoire σ des fenêtres en XY (µm) : erreur de position simulée de l'étage 1
`--jitter_z`	0.5	idem en Z (µm)
`--vmin`	20	voxels self minimum pour garder une détection
`--base`	16	canaux du premier niveau (doublés à chaque niveau)
`--depth`	3	niveaux du U-Net (sous-échantillonnages par 2)
`--eval_every`	500	validation toutes les N itérations
`--val_limit`	12	hologrammes de validation utilisés
`--seed`	0	graine

Règles et conseils

- **Décimation et filtre** : la décimation recadre le spectre ; sans perte si le passe-bas est ≤ taille / (2 × décimation) : 256 px pour 1024 et ×2, 128 pour ×4. Les deux étages utilisés ensemble doivent avoir la même décimation (l'inférence refuse sinon).
- **Tailles multiples de 2^niveaux** (8 pour 3 niveaux) : patchs de l'étage 1, fenêtre fine de l'étage 2 (plans et côté). Le carré refocalisé doit contenir la fenêtre fine : côté fin ≤ ¾ × décimation × carré.
- **Normalisation** : filmmad (défaut du GUI) — mesurée sur le film, constante d'une image à l'autre. Au benchmark du 22 octobre, toutes les normalisations se valent à nombre de détections égal (resultats/benchmark_grid_B/RAPPORT_NORMALISATION_ETAGE1.pdf).
- **Profondeur** : vide = celle du jeu ; au moins la profondeur des bactéries des films à analyser.
- **Seuil de l'étage 1** : choisi à la fin de l'entraînement (meilleur F1 en validation simulée) et enregistré ; sur un film réel, il se règle à l'inférence (HELP_INFERENCE.pdf).
- **Mémoire** (Quadro RTX 3000, 6 Go) : patchs 48 × 96 × 96 par 4 ≈ 5 Go ; 64 × 128 × 128 déborde. Une décimation de 1 multiplie la mémoire et le temps de reconstruction par 4.
- **Réentraîner à l'identique** : même jeu, mêmes options, même graine (--seed).